PROJET DE FIN DE MODULE · AWS CLOUD ARCHITECTING (M1)

AfricaShop

RAPPORT DE CONCEPTION ET DE DÉPLOIEMENT SUR AWS

Place de marché haute disponibilité pour l'artisanat d'Afrique de l'Ouest

Étudiant Diene Sarr

Enseignant M. Baye Sabarane LAM

Région AWS us-east-1 (Virginie du Nord)

Date de remise 31 octobre 2026

Plan du rapport

Introduction 2
1. Conception et planification 2
1.1 Diagramme d'architecture 2
1.2 Estimation des coûts avec AWS Pricing Calculator 5
2. Déploiement et configuration 6
2.1 Mise en place du réseau AWS 6
2.2 Déploiement du serveur web et de l'API 7
2.3 Mise en place de la base de données 8
2.4 Automatisation du déploiement 9
3. Sécurisation et tests 10
3.1 Configuration IAM, authentification et permissions 10
3.2 Tests de l'application 11
4. Analyse des défis et solutions 14
Conclusion 14
Limites et perspectives vers la production 14
Suppression des ressources 15
Annexes 16

Introduction

AfricaShop est une place de marché en ligne qui relie des artisans et des commerçants d'Afrique de l'Ouest (wax, bogolan, vannerie de Thiès, beurre de karité, café Touba) à des acheteurs du monde entier. Le client, M. LAM, demande une plateforme disponible 24 h/24 (99,9 % par mois), capable d'absorber 100 000 visiteurs simultanés, rapide sur mobile, avec envoi de photos par les vendeurs et données clients protégés, pour un budget de 50 000 USD la première année.

Dans ce projet, j'ai diagnostiqué l'application de départ (AfricaShop Starter), conçu une architecture AWS multi-zones puis je l'ai déployée dans la région **us-east-1**. Ce rapport détaille la conception, le déploiement, la sécurisation et les tests, ainsi que les défis rencontrés. Les services utilisés sont : **VPC, EC2 Auto Scaling, Application Load Balancer, RDS PostgreSQL Multi-AZ, S3, CloudFront, WAF, Cognito, IAM, Secrets Manager, CloudWatch et CloudTrail**. Le déploiement a été fait par la console AWS et par des commandes CLI dans CloudShell ; le code est sur GitHub et récupéré par les instances au démarrage.

1. Conception et planification

1.1 Diagramme d'architecture

Le starter concentre sept faiblesses qui contredisent les exigences du client. Chaque composant de l'architecture cible répond à l'une d'elles.

#	Faiblesse du starter	Exigence	Réponse dans l'architecture cible
1	Une instance, une zone	NF-01 disponibilité	Deux zones, ALB, Auto Scaling group, RDS Multi-AZ
2	Instance publique, port 80, pas de HTTPS	NF-05 sécurité	Instances et base en sous-réseaux privés, HTTPS par CloudFront, groupes de sécurité chaînés
3	Base SQLite sur le disque	NF-05, NF-07	RDS PostgreSQL Multi-AZ, sauvegardes de 7 jours
4	Photos sur le disque, via le serveur	NF-04, NF-08	S3 et URL pré-signées : upload direct du navigateur
5	Ni sauvegarde, ni alarme	NF-07, NF-10	CloudWatch, CloudTrail, sauvegardes RDS
6	Secret de paiement connu du front	NF-06	Paiement délégué à un prestataire PCI DSS (simulé ici)
7	Profil d'instance trop permissif	NF-05	Rôle IAM dédié à moindre privilège

Les utilisateurs accèdent à la plateforme par une seule adresse HTTPS **Amazon CloudFront**, protégée par **AWS WAF**. Selon le chemin demandé, CloudFront lit le bucket S3 du site statique, transmet /api/* à l'ALB, ou sert les photos depuis un second bucket. Derrière l'ALB, des instances EC2 sans adresse publique exécutent l'API dans un conteneur Docker ; la base RDS est dans des sous-réseaux privés dédiés.

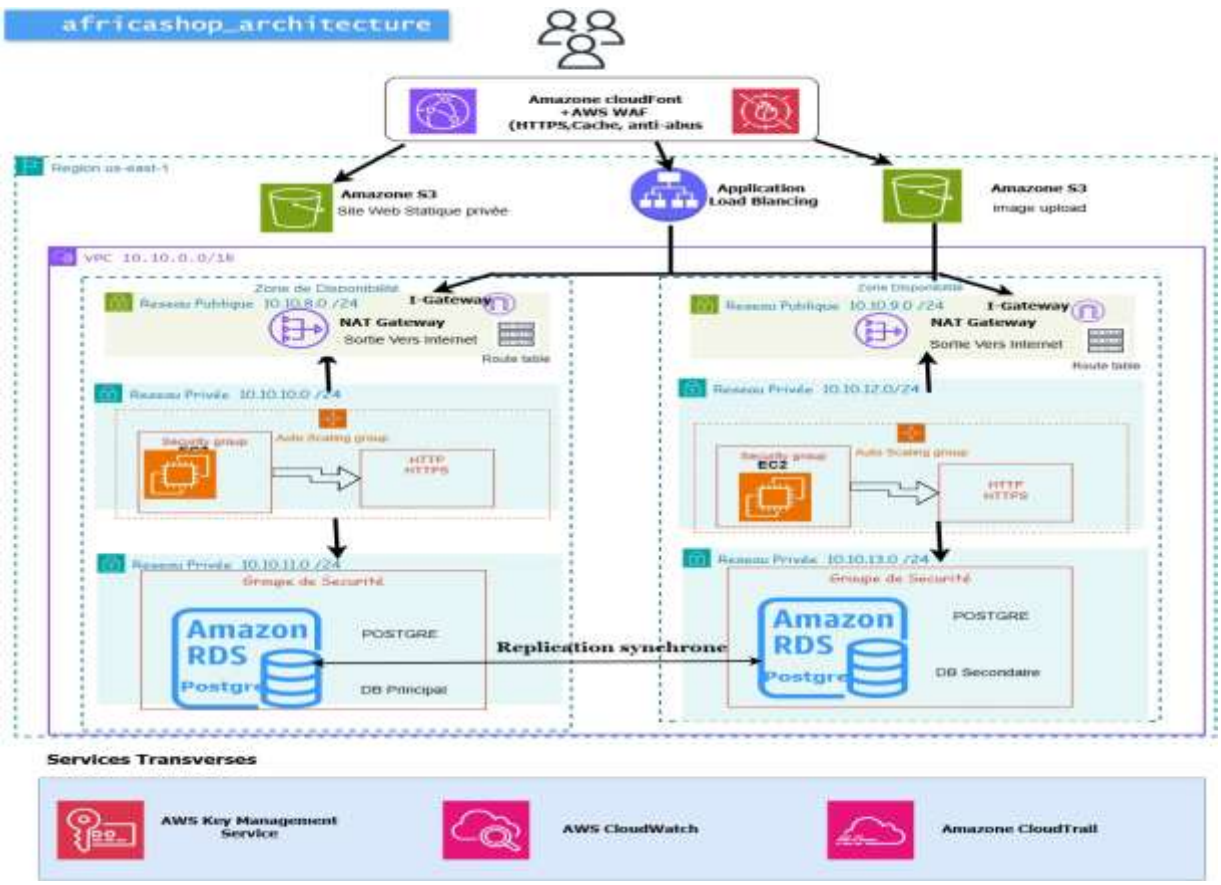


Figure 1 : *Architecture cible d'AfricaShop : CloudFront et WAF, deux zones, ALB en sous-réseaux publics, instances et base en sous-réseaux privés, services transverses.*

Flux de données

1. Le visiteur ouvre l'URL CloudFront en HTTPS ; le WAF inspecte la requête.
2. Pages et scripts : CloudFront lit le bucket du site (privé) par Origin Access Control.
3. Appels /api/* : CloudFront les transmet à l'ALB, sans cache.
4. L'ALB choisit une instance saine dans l'une des deux zones (port 8000).
5. L'API lit son mot de passe dans Secrets Manager puis interroge RDS (port 5432).
6. Photos : l'API délivre une URL pré-signée, le navigateur dépose la photo dans S3, servie ensuite par /media/*.
7. Les instances sortent vers Internet par les NAT Gateways et joignent S3 par un point de terminaison de passerelle.

Dimensionnement et région

Par la loi de Little ($\lambda = L / W$, avec L = 100 000 visiteurs et W = 15 s), CloudFront voit environ 6 700 requêtes par seconde. Avec 80 % de cache, l'origine en reçoit environ 1 300, soit 6 instances à 250 requêtes par seconde, et 8 à 10 avec une marge de 50 %. Ce sont des hypothèses à confronter au test de charge (section 3.2). Pour la production, la région **Paris (eu-west-3)** est recommandée (latence vers l'Europe et l'Afrique de l'Ouest par CDN, RGPD) ; le projet utilise us-east-1, contrainte de l'environnement de lab.

Exigences retenues et traduction technique

Exigence du client	Traduction technique
Disponibilité 99,9 % et survie à la perte d'une zone (NF-01)	Deux zones, ALB sur les deux, Auto Scaling group sur deux sous-réseaux, RDS Multi-AZ
Rapidité et pointes de trafic (NF-02, NF-03)	CloudFront devant le site, les photos et l'API ; cache pour le contenu statique et les photos
Upload de photos sans charger l'API (NF-04)	URL pré-signées S3 : le navigateur dépose la photo directement dans le bucket
Protection des données (NF-05)	Aucune ressource privée exposée, chiffrement au repos, groupes de sécurité chaînés, secrets gérés, moindre privilège
RPO 15 min, RTO 1 h pour une panne de zone (NF-07)	Multi-AZ synchrone et sauvegardes automatiques de 7 jours
Élasticité et exploitabilité (NF-08, NF-10)	Auto Scaling de 2 à 6 instances, modèle de lancement, alarmes, tableau de bord, CloudTrail

Questions au client et hypothèses retenues

Question posée au client	Hypothèse retenue faute de réponse
Les 100 000 visiteurs de pointe sont-ils anonymes ? Quelle conversion ?	Visiteurs majoritairement anonymes, conversion faible : trafic dominant = lecture du catalogue, donc cachable
La croissance de 200 % signifie-t-elle ×2 ou ×3 ?	×3 sur douze mois
Les données doivent-elles rester au Sénégal ou sur le continent ?	Pas de contrainte stricte pour le pilote ; à valider auprès de la CDP avant l'ouverture commerciale
Quel prestataire de paiement est sous contrat, et qui exploite la nuit ?	Paiement simulé (prestataire PCI DSS à contractualiser) ; exploitation automatisée, astreinte à définir

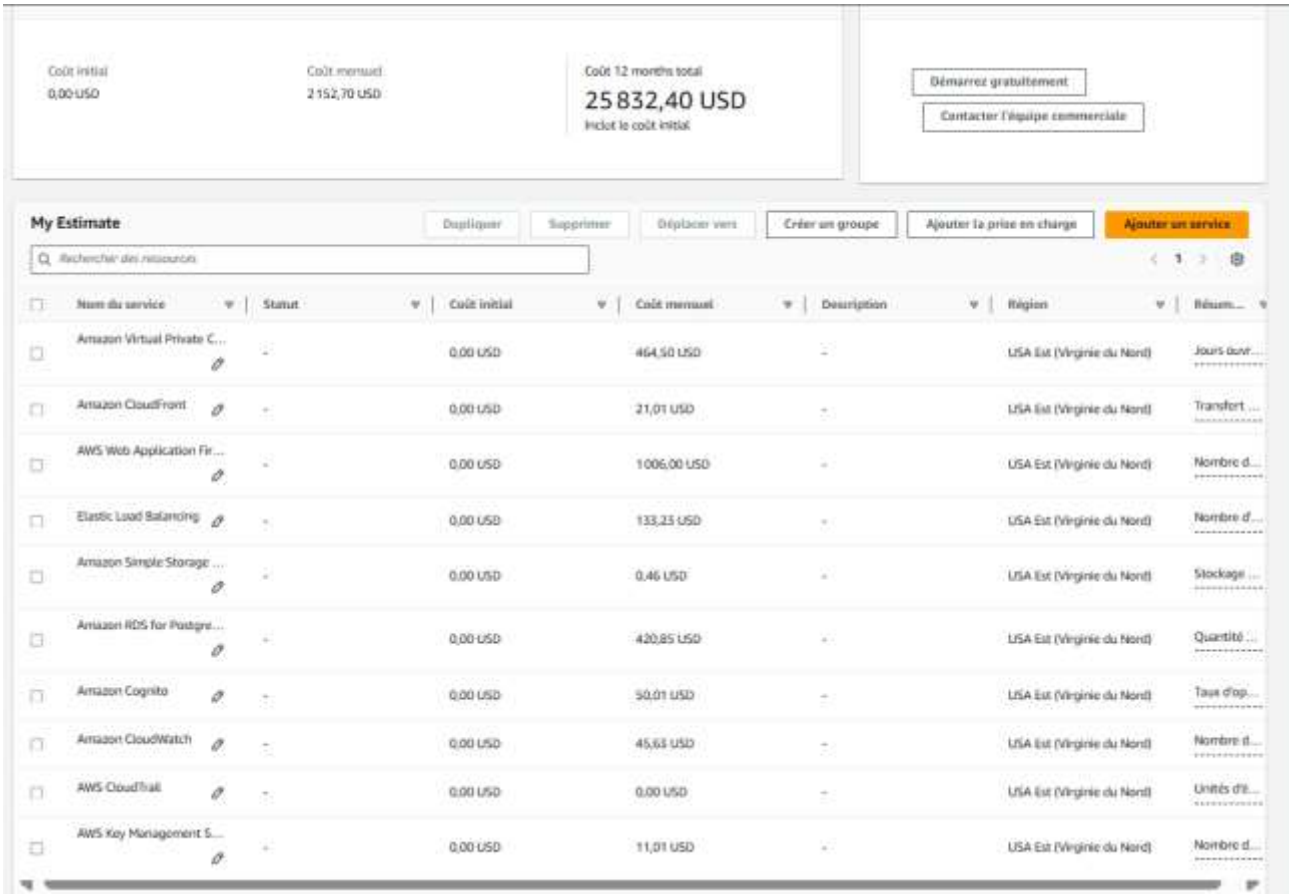
Services AWS retenus et justification

Service	Rôle dans AfricaShop	Alternative écartée et pourquoi
VPC sur 2 zones	Isolation, sous-réseaux publics et privés	VPC par défaut du starter : tout y est public
CloudFront	HTTPS, cache, point d'entrée unique	Accès direct à l'ALB : ni cache ni protection en bordure
AWS WAF	Règles gérées, limitation de débit	Aucune protection applicative
Application Load Balancer	Répartition entre zones, santé sur /api/health	Network Load Balancer : pas de routage HTTP ni de santé applicative
EC2 Auto Scaling	Élasticité de 2 à 6 instances, remplacement automatique	Instance unique : aucune tolérance aux pannes
RDS PostgreSQL Multi-AZ	Commandes, vendeurs, produits ; standby synchrone	DynamoDB : le modèle relationnel (commande, lignes, stock) est plus simple en SQL ; SQLite : disque perdu avec l'instance
Amazon S3	Site statique et photos, buckets privés	Disque des instances : perdu avec l'instance

Service	Rôle dans AfricaShop	Alternative écartée et pourquoi
Secrets Manager	Mot de passe de la base géré par RDS	Variable en clair : visible et jamais renouvelée
Amazon Cognito	Authentification des vendeurs par jeton signé	Mots de passe gérés par l'API : risque et charge
KMS	Chiffrement au repos (RDS)	Données non chiffrées
CloudWatch et CloudTrail	Alarmes, tableau de bord, journal des actions	Surveillance manuelle, aucune traçabilité
Systems Manager	Accès aux instances sans SSH	SSH et clés : surface d'attaque supplémentaire

1.2 Estimation des coûts avec AWS Pricing Calculator

L'estimation de production se fait dans AWS Pricing Calculator avec les hypothèses du sujet : 1,5 To servi par le CDN et 30 millions de requêtes par mois au lancement, croissance ×3 en douze mois, 100 Go de photos puis 300 Go. Les montants ne sont pas renseignés tant que le calculateur n'a pas été rempli : aucun chiffre n'est avancé sans source.voici le lien sur princing calculator : https://calculator.aws/#/estimate?id=6fff6eee148b1c16467bf2f65204f118db6e3da9



Leviers d'optimisation : point de terminaison S3 (évite le traitement NAT sur le trafic S3), instances Graviton (environ 20 % moins chères), Savings Plans après quelques mois de mesure, cache CloudFront maximal, S3 Intelligent-Tiering pour les photos, alerte de budget à 80 %. Les NAT Gateways et RDS Multi-AZ facturent à l'heure même sans trafic : ce sont les postes à surveiller.

2. Déploiement et configuration

2.1 Mise en place du réseau AWS

Le VPC utilise le bloc **10.10.0.0/16** sur deux zones de disponibilité, avec six sous-réseaux /24 : deux publics (ALB et NAT Gateways), deux privés applicatifs (EC2) et deux privés pour les données (RDS). Une passerelle Internet dessert les sous-réseaux publics ; chaque zone a sa NAT Gateway et sa table de routage privée, si bien que la perte d'une zone ne coupe pas la sortie Internet de l'autre. Un point de terminaison de passerelle S3 évite le coût de traitement des NAT sur le trafic S3.

Rôle	us-east-1a	us-east-1b	Groupe de sécurité	Entrée autorisée
Public (ALB, NAT)	10.10.8.0/24	10.10.9.0/24	sg_alb	HTTP 80 depuis la liste de préfixes CloudFront
Privé applicatif (EC2)	10.10.10.0/24	10.10.12.0/24	sg_app	TCP 8000 depuis sg_alb uniquement
Privé données (RDS)	10.10.11.0/24	10.10.13.0/24	sg_db	PostgreSQL 5432 depuis sg_app uniquement

Les trois groupes forment une chaîne : chacun n'accepte que le groupe qui le précède. Il n'existe aucune règle SSH ; l'accès aux machines passe par Session Manager.

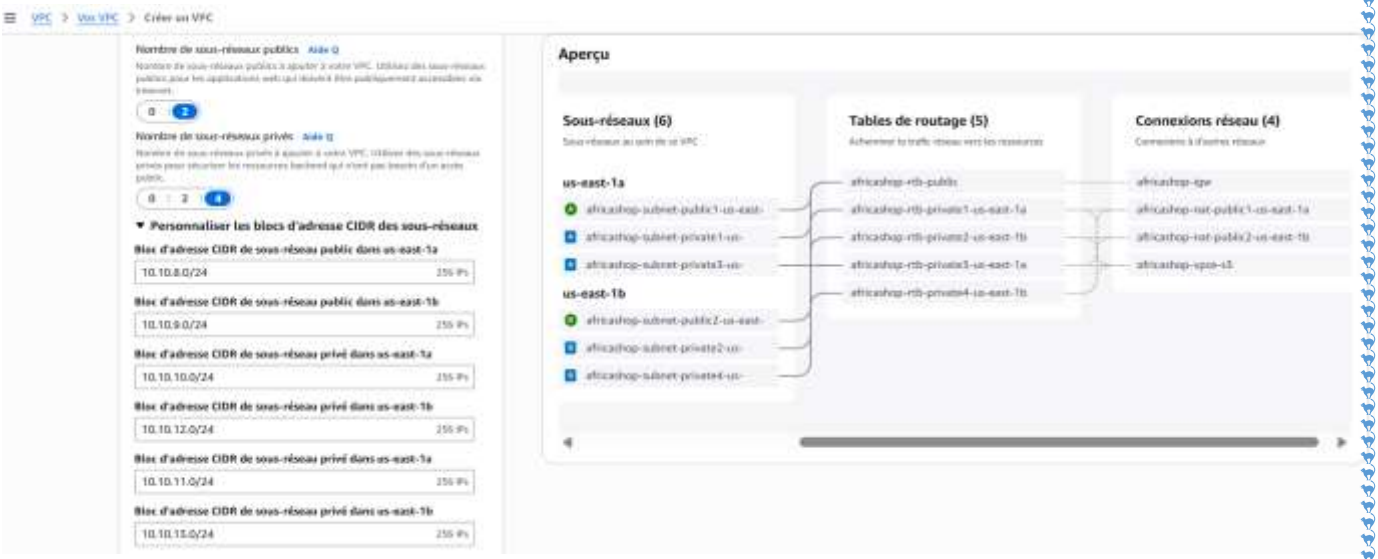


Figure 2 : *Assistant de création du VPC : 6 sous-réseaux, 5 tables de routage, passerelle Internet, 2 NAT Gateways et point de terminaison S3.*

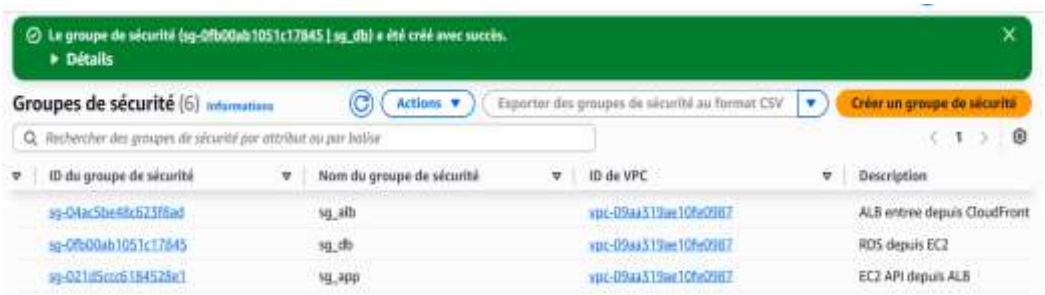


Figure 3 : *Groupes de sécurité sg_alb, sg_app et sg_db du VPC du projet.*

2.2 Déploiement du serveur web et de l'API

Le **modèle de lancement** décrit l'instance : Amazon Linux 2023, type t3.small, groupe sg_app, profil IAM du projet, IMDSv2 obligatoire (limite de sauts portée à 2 pour que le conteneur obtienne les identifiants du rôle) et un script de démarrage (annexe B) qui installe Docker, clone le dépôt, écrit la configuration puis construit et lance l'image de l'API. L'**ALB**, exposé à Internet dans les deux sous-réseaux publics, contrôle la santé des cibles sur /api/health, qui vérifie aussi la base. Le **groupe Auto Scaling** africashop-asg répartit 2 à 6 instances sur les deux sous-réseaux applicatifs, avec un suivi de cible à 50 % de CPU, un contrôle de santé de l'ALB et une période de grâce de 300 secondes.

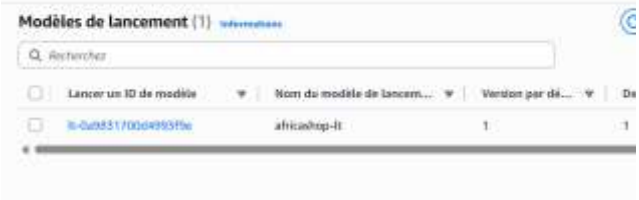


Figure 4 : *Modèle de lancement africashop-lt.*

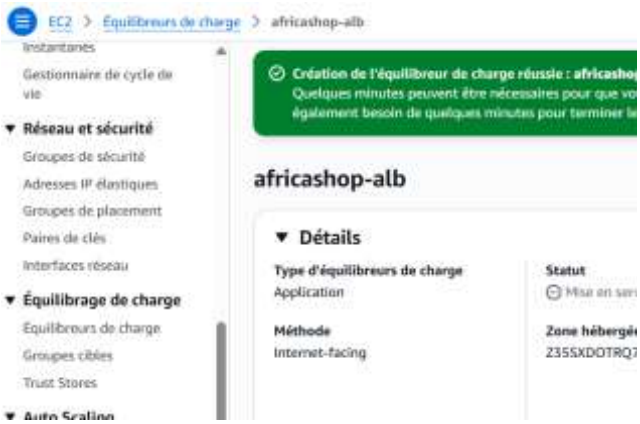


Figure 5 : *Application Load Balancer africashop-alb (Internet-facing).*

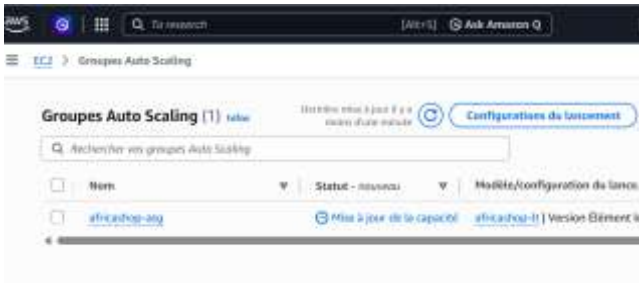


Figure 6 : *Groupe Auto Scaling africashop-asg.*



Figure 7 : *Deux instances t3.small lancées par le groupe.*

Le site statique est dans un bucket S3 **privé**, lu uniquement par CloudFront (Origin Access Control) ; l'accès direct au bucket renvoie AccessDenied. CloudFront redirige HTTP vers HTTPS et définit index.html comme objet racine. La distribution a été créée avec **AWS WAF** en **mode surveillance** : les requêtes qui seraient bloquées sont comptées, ce qui permet d'observer les faux positifs avant le passage en blocage.

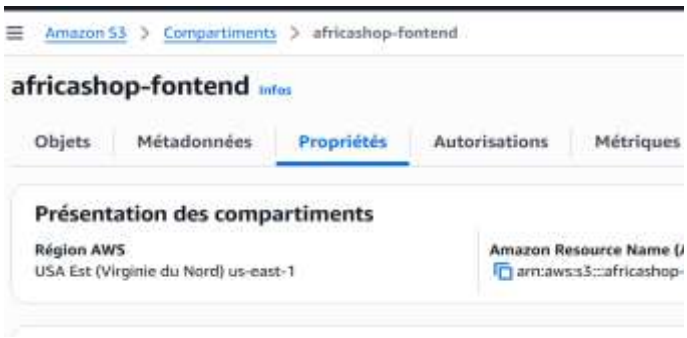


Figure 8 : *Bucket du site statique (us-east-1), privé.*



Figure 9 : *Protection WAF avec mode de surveillance coché.*

L'API FastAPI est servie sous /api/* : plateforme (/api/health, /api/info qui indique l'instance et la zone qui répondent), catalogue, espace vendeur (dont l'URL pré-signée pour les photos) et commandes. La documentation interactive est sous /api/docs.

Principes de conception

- **Rien de privé n'est exposé** : seuls CloudFront et l'ALB sont joignables depuis Internet, et l'ALB n'accepte que CloudFront.
- **Instances jetables et sans état** : l'état est dans RDS et S3, une instance perdue est remplacée sans intervention.
- **Moindre privilège à chaque couche** : groupes de sécurité chaînés, rôle IAM minimal, accès par Session Manager.
- **Configuration par variables d'environnement** : le même code fonctionne en local et sur AWS sans modification.

2.3 Mise en place de la base de données

La base est une instance **Amazon RDS pour PostgreSQL 17** en **Multi-AZ**, dans le groupe de sous-réseaux de données et protégée par sg_db. Elle est chiffrée au repos (clé AWS KMS gérée), son stockage est un SSD gp3 de 200 Gio et ses sauvegardes automatiques sont conservées 7 jours. Multi-AZ ne crée pas deux bases : AWS maintient un standby synchrone, non joignable, dans l'autre zone (ici us-east-1a, la principale étant en us-east-1b) et bascule automatiquement, avec le même point d'accès. Le mot de passe principal est généré par RDS et conservé dans **Secrets Manager** (secret actif) : il n'apparaît dans aucun fichier ni dans Git, et l'API le lit au démarrage grâce à son rôle IAM.

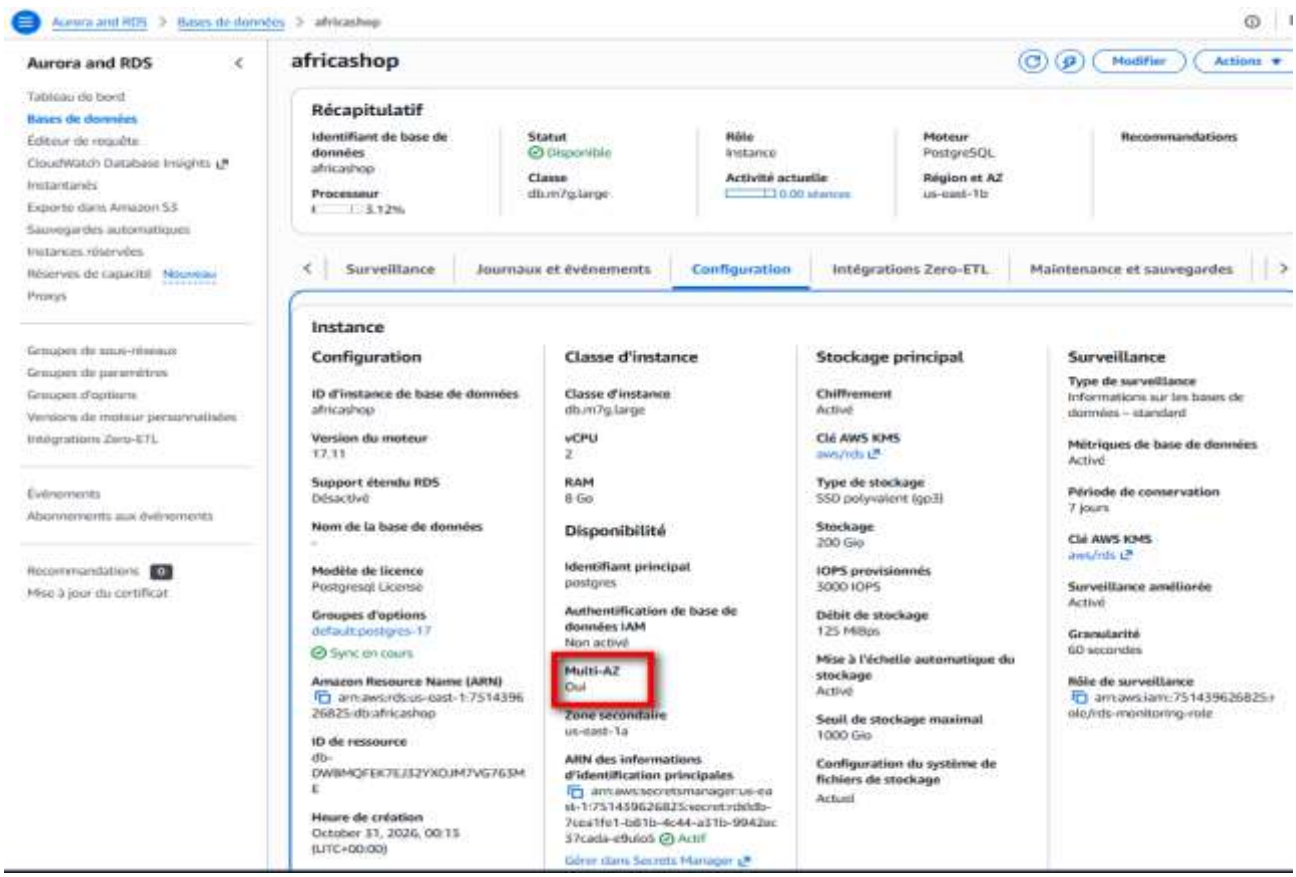


Figure 10 : *Instance RDS africashop : PostgreSQL 17.11, Multi-AZ « Oui » (zone secondaire us-east-1a), chiffrement activé, rétention 7 jours, secret géré dans Secrets Manager.*

Dimensionnement. La classe db.m7g.large (2 vCPU, 8 Go) et les 200 Gio de stockage sont généreux pour un pilote : RDS facture à l'heure, même sans trafic. Une classe plus petite (db.t4g.micro ou small) suffit pour les essais, et la classe de production sera choisie d'après le test de charge.

La base logique africashop n'avait pas été demandée à la création du serveur ; elle a été créée ensuite par un conteneur ponctuel qui lit le secret, sans exposer le mot de passe (voir section 4). Les tables sont ensuite créées et remplies par le script d'initialisation de l'application.

2.4 Automatisation du déploiement

Pour que le redéploiement soit reproductible, la partie calcul est pilotée par des scripts AWS CLI versionnés avec le code. Un premier script lit lui-même l'endpoint RDS et l'ARN du secret (plus de saisie manuelle, donc plus d'erreur de copie), crée une **nouvelle version du modèle de lancement** avec le script de démarrage, l'applique au groupe Auto Scaling et lance une **actualisation des instances** : les instances sont remplacées une à une, avec au moins 50 % de capacité saine pendant l'opération. Un second script ajoute à CloudFront l'**origine ALB** et le comportement /api/* (cache désactivé, toutes les méthodes HTTP, redirection vers HTTPS) puis invalide le cache.

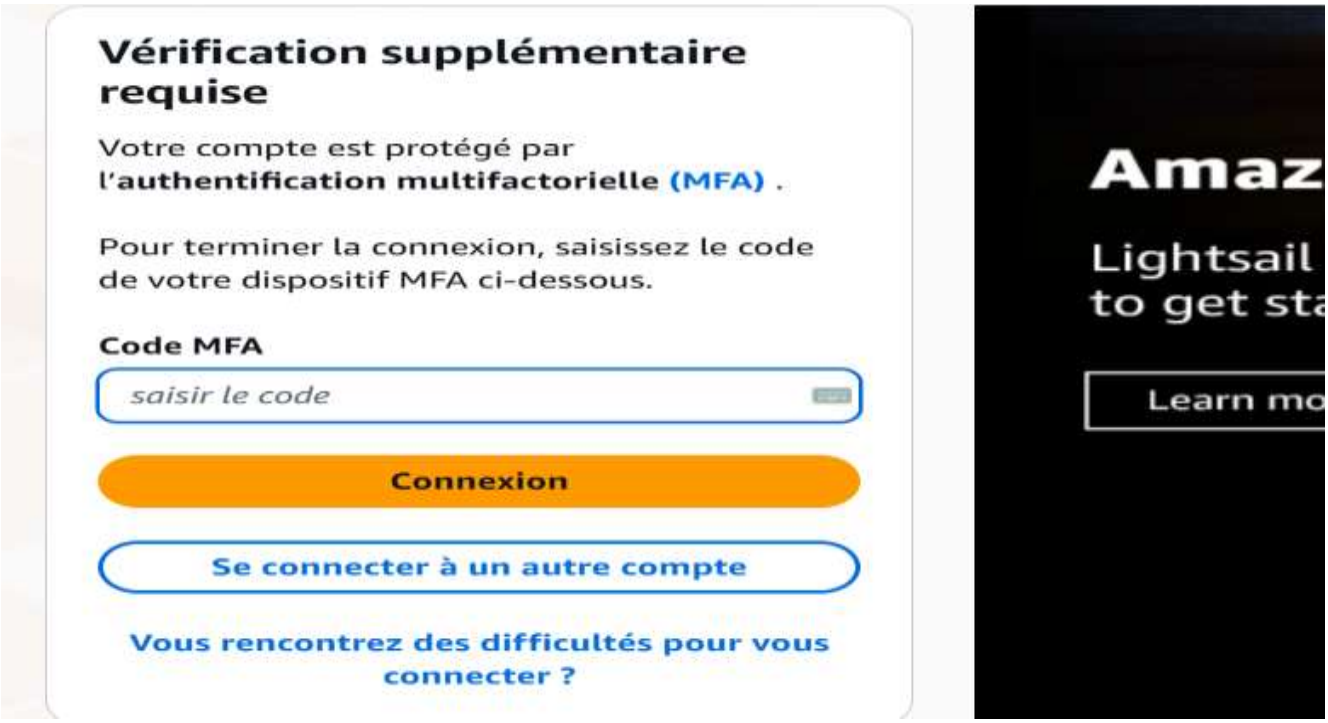
```
aws ec2 create-launch-template-version --launch-template-name africashop-lt --source-version '$Latest' ...
aws autoscaling update-auto-scaling-group --auto-scaling-group-name africashop-asg --launch-template ...
aws autoscaling start-instance-refresh --auto-scaling-group-name africashop-asg \
  --preferences '{"MinHealthyPercentage":50,"InstanceWarmup":300}'
aws cloudfront update-distribution --id <id-distribution> --distribution-config file://cf-new.json --if-match <etag>
```

Ces scripts sont une étape vers l'infrastructure as code : le réseau, la base et la sécurité ont été créés par la console, et leur transcription en CloudFormation est la suite logique (voir Conclusion).

3. Sécurisation et tests

3.1 Configuration IAM, authentification et permissions

Le travail est fait avec un **utilisateur IAM dédié** plutôt qu'avec le compte racine. Cet utilisateur a des droits étendus, acceptables sur un compte de projet jetable ; la politique de moindre privilège pour la production est fournie en annexe A.



Les instances, elles, n'ont que le **rôle africashop-ec2-role** : lecture d'un seul secret (celui de la base), écriture et lecture dans le préfixe products/ du bucket des photos **sans aucune suppression**, plus Session Manager et l'agent CloudWatch. Le filtrage est réel : depuis une instance, une commande de lecture de la configuration RDS est refusée (AccessDenied), alors qu'elle réussit avec l'utilisateur d'administration.



Figure 11 : *Création de l'utilisateur IAM dédié (champ du mot de passe retiré).*

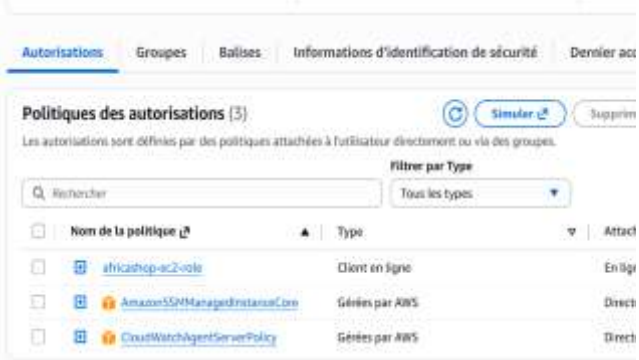


Figure 12 : *Autorisations du rôle des instances : politique en ligne limitée et deux politiques gérées.*

Domaine	Mesure mise en place
Réseau	Aucune instance ni base publique ; ALB limité à CloudFront ; chaîne sg_alb → sg_app → sg_db ; pas de SSH
Chiffrement	RDS chiffrée (KMS) ; buckets S3 en SSE-S3 ; HTTPS entre le visiteur et CloudFront ; CloudFront → ALB en HTTP (limite, section Conclusion)
Secrets	Mot de passe de base géré par RDS dans Secrets Manager ; aucun secret dans le dépôt ; IMDSv2 obligatoire
Application	WAF en surveillance, à passer en blocage avec limitation de débit ; paiement délégué à un prestataire PCI DSS (SAQ A) en production, simulé dans le starter
Données personnelles	Loi sénégalaise n° 2008-12 (CDP) et RGPD : déclaration des traitements et encadrement des transferts à prévoir avant l'ouverture commerciale

Authentification des vendeurs avec Amazon Cognito. L'API est passée en mode cognito par trois variables d'environnement. Elle vérifie la signature du jeton d'accès, le client émetteur et l'appartenance au groupe vendeurs, et lie chaque boutique à l'identifiant Cognito de son propriétaire : un vendeur ne modifie que sa boutique. Le mot de passe ne transite jamais par les serveurs d'AfricaShop. Le frontend du starter attendait une page de connexion qui n'existait pas : une page login.html a été ajoutée. Deux points d'attention : l'auto-inscription, proposée par défaut, doit être désactivée en production, et le compte vendeur créé par l'administrateur affichait le statut « Forcer la modification du mot de passe », qui bloque la connexion tant qu'un mot de passe permanent n'est pas défini.



Figure 13 : *Application Cognito de type page unique, connexion par adresse e-mail.*

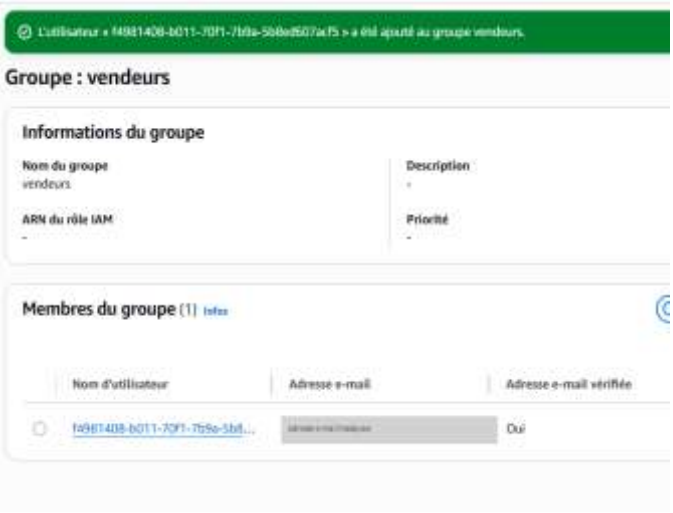


Figure 14 : *Groupe « vendeurs » et son premier membre (e-mail masqué).*

3.2 Tests de l'application

Les tests vérifient chaque exigence par une preuve observable. Les deux captures ci-dessous prouvent la répartition entre zones : le bandeau du site indique l'instance qui a répondu et sa zone. Sur la première, l'instance

affiche encore auth none : Cognito avait d'abord été activé sur une seule instance pour être testé, puis appliqué à toutes par le modèle de lancement.

Figure 15 : *Réponse servie par une instance de us-east-1a.*



Figure 16 : *Réponse servie par une instance de us-east-1b, Cognito actif.*



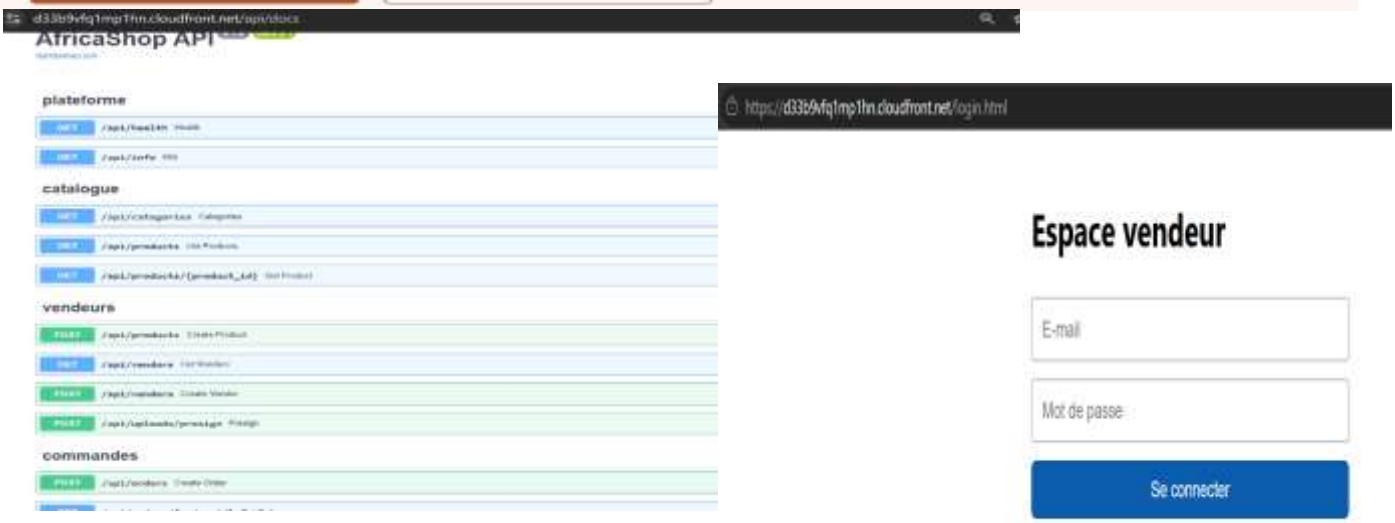


Figure 17 : *Documentation de l'API servie à travers CloudFront (/api/docs).*

Protocole du test de restauration RDS

1. Noter l'heure et créer une donnée repérable (une commande de test).
2. Restaurer l'instance à un instant antérieur dans une nouvelle instance RDS.
3. Chronométrer jusqu'au statut Disponible et vérifier la présence des données attendues.
4. Calculer le RPO observé (écart avec le dernier enregistrement retrouvé) et le RTO (durée de restauration), puis supprimer l'instance restaurée.

Protocole du test de charge (Locust)

300 utilisateurs virtuels pendant 10 minutes, avec un mélange de lectures du catalogue et de quelques écritures. Le but n'est pas de simuler les 100 000 visiteurs mais de mesurer la capacité réelle d'une instance (requêtes par seconde à 70 % de CPU), le nombre d'instances atteint, le délai de montée en charge et la latence p95, pour corriger les hypothèses de la section 1.1. Un point est déjà connu : une nouvelle instance met 4 à 5 minutes à devenir saine car l'image Docker est construite au démarrage ; avant un pic annoncé, la capacité minimale sera relevée à l'avance.

Sauvegarde et reprise. Panne d'une zone : bascule automatique Multi-AZ (cible RPO 15 min, RTO 1 h). Erreur humaine : restauration à un instant donné depuis les sauvegardes de 7 jours. Perte de région : copie de sauvegardes inter-régions (cible RPO 24 h, RTO 8 h), décrite mais non testée. Les instances sont reproductibles (modèle de lancement et Git) et les photos sont dans S3.

Supervision et traçabilité

Signal	Métrique CloudWatch	Seuil d'alarme	État
Trafic	Nombre de requêtes de l'ALB	Tableau de bord	OK
Erreurs	Erreurs 5xx des cibles de l'ALB	Plus de 5 sur 5 minutes	OK
Latence	Temps de réponse des cibles	p95 inférieure à 500 ms	OK
Saturation	CPU du groupe Auto Scaling et de RDS	CPU RDS supérieur à 80 %	OK

AWS CloudTrail journalise les appels d'API du compte dans un bucket S3 dédié : il permet de savoir qui a modifié quoi et quand, ce qui est indispensable pour une enquête de sécurité.



Figure 19 : *Bucket des journaux CloudTrail (préfixe AWSLogs).*

4. Analyse des défis et solutions

Défi rencontré	Cause	Solution
Erreur 403 sur le site derrière CloudFront	Origine « site web statique » de S3, incompatible avec OAC et un bucket privé ; objet racine non défini	Origine S3 REST avec OAC, index.html comme objet racine, invalidation du cache
Accès impossible aux instances par Session Manager	Instances démarrées avant l'association effective du rôle au profil	Vérification du profil et des politiques, remplacement des instances par le groupe Auto Scaling
Le script de démarrage s'arrêtait au clonage	Dépôt GitHub privé : clonage anonyme refusé, arrêt sur erreur du script	Dépôt rendu public après contrôle de l'absence de secret ; clonage sur une seule ligne
L'API échouait à lire le secret (Invalid name)	DB_SECRET_ARN contenait l'ARN de la base RDS et non celui du secret	ARN du secret récupéré par script ; fichier réécrit sans interprétation du caractère « ! »
L'API échouait à joindre la base (database does not exist)	Base logique africashop non créée avec le serveur RDS	Création par un conteneur ponctuel lisant le secret, sans exposer le mot de passe
Conteneur en redémarrage permanent	Anciennes variables conservées : elles ne sont lues qu'à la création	Suppression et recréation du conteneur avec le fichier corrigé
Commandes RDS refusées depuis l'instance	Mauvais terminal : le rôle de l'instance n'a volontairement aucun droit sur RDS	CloudShell pour l'administration, Session Manager pour l'instance. Le refus valide le moindre privilège

Conclusion

L'architecture déployée répond aux sept faiblesses du starter : deux zones de disponibilité, instances et base en sous-réseaux privés derrière CloudFront et un ALB, PostgreSQL géré en Multi-AZ, photos dans S3 par URL pré-signée, secrets gérés, rôle IAM à moindre privilège, authentification Cognito et traçabilité CloudTrail. La répartition entre deux zones est démontrée par le bandeau de l'application. Il reste à finaliser les mesures qui conditionnent les engagements chiffrés : restauration de la base, test de charge, alarmes et estimation des coûts.

Limites et perspectives vers la production

Sujet	Situation actuelle	Amélioration
CloudFront vers ALB	HTTP, sans domaine ni certificat	Domaine Route 53 et certificat ACM : HTTPS de bout en bout
Image applicative	Build Docker à chaque démarrage (4 à 5 min)	Image dans Amazon ECR, version figée par étiquette
Infrastructure as code	Console, CLI et scripts	CloudFormation ou Terraform, redéploiement en une commande (exigence du sujet : principal écart de ce rendu)

Sujet	Situation actuelle	Amélioration
Paiement, WAF	Paiement simulé ; WAF en surveillance	Prestataire PCI DSS ; WAF en blocage et limitation de débit
Région et reprise	us-east-1 ; reprise inter-régions non testée	eu-west-3 (Paris) ; copie de sauvegardes inter-régions

Plan progressif : pilote à 2 instances, puis lancement (domaine, HTTPS, paiement réel, alarmes complètes, test de charge validé), puis croissance (capacité relevée avant les pics, ECR), puis année 2 (SNS/SQS, plan de reprise inter-régions).

Suppression des ressources

Une fois les preuves capturées, tout est supprimé dans l'ordre : groupe Auto Scaling, ALB et groupe cible, distribution CloudFront (désactivée puis supprimée), RDS sans sauvegarde finale, NAT Gateways et adresses IP élastiques, buckets S3 vidés puis supprimés, VPC et groupes de sécurité, Cognito, secrets, alarmes, trail et rôles.

Déclaration d'usage des assistants IA

Un assistant d'IA (Claude, d'Anthropic) a servi à analyser le sujet et les schémas, guider la configuration des services, diagnostiquer les erreurs, rédiger des commandes et scripts. Les commandes ont été exécutées et vérifiées par moi-même, et je peux expliquer paramètre par paramètre en soutenance. Les captures et mesures proviennent de mon environnement.

Annexes

Annexe A : politique IAM du rôle des instances (moindre privilège)

```
{ "Version": "2012-10-17", "Statement": [
  { "Sid": "LireSecretBase", "Effect": "Allow",
    "Action": "secretsmanager:GetSecretValue",
    "Resource": "arn:aws:secretsmanager:us-east-1:<compte>:secret:rds!db-*" },
  { "Sid": "PhotosProduits", "Effect": "Allow",
    "Action": ["s3:PutObject", "s3:GetObject"],
    "Resource": "arn:aws:s3:::africashop-media-prod/products/*" } ] }
```

Politiques gérées ajoutées : AmazonSSMManagedInstanceCore et CloudWatchAgentServerPolicy. Aucune action de suppression sur S3.

Annexe B : script de démarrage des instances (sans mot de passe)

```
#!/bin/bash
set -euxo pipefail
dnf install -y docker git && systemctl enable --now docker
git clone --depth 1 https://github.com/Dienesarr/Africashop_Projet_AWS_Deploy.git /opt/africashop
cat > /etc/africashop.env <<'ENV'
APP_ENV=aws
DB_HOST=<endpoint-rds>
DB_SECRET_ARN=<arn-du-secret-secrets-manager>
STORAGE_MODE=s3
MEDIA_BUCKET=africashop-media-prod
MEDIA_PUBLIC_BASE=/media
SERVE_FRONTEND=false
CORS_ORIGINS=https://<domaine-cloudfront>
AUTH_MODE=cognito
COGNITO_USER_POOL_ID=<id-groupe>  COGNITO_CLIENT_ID=<id-client>
ENV
cd /opt/africashop/backend && docker build -t africashop-api .
docker run -d --name api --restart always --env-file /etc/africashop.env -p 8000:8000 africashop-api
```